HiveCollective — Project Outline

Vision

A collective intelligence platform where communities across Africa and the developing world name the problems that hold them back — and build solutions together. Not a complaint board. A structured engine for turning collective frustration into collective action.

Target users: Individuals, community groups, policy-makers, and researchers. **Languages:** English, French, Spanish. **Scope:** Problems tagged at local (city/village) or national (country) level.

Tech Stack

Layer	Technology
Frontend	React + Vite
Backend	Node.js + Express
Database	PostgreSQL
Auth	bcrypt + JWT
Deployment (planned)	Vercel (frontend), Railway (backend + DB)

Database Schema

users

Column	Type	Notes
id	SERIAL PK	
username	VARCHAR(50)	Unique
email	VARCHAR(255)	Unique
password_hash	TEXT	bcrypt

Column	Type	Notes
preferred_language	VARCHAR(5)	en, fr, es
created_at	TIMESTAMPTZ	

problems

Column	Type	Notes
id	SERIAL PK	
user_id	INTEGER FK	→ users
title	VARCHAR(255)	
description	TEXT	
scope	ENUM	local, national
location_tag	VARCHAR(100)	e.g. Ghana, Accra
created_at	TIMESTAMPTZ	

solutions

Column	Type	Notes
id	SERIAL PK	
problem_id	INTEGER FK	→ problems
user_id	INTEGER FK	→ users
content	TEXT	Max 5000 chars
score	INTEGER	Updated on each vote
created_at	TIMESTAMPTZ	

votes

Column	Type	Notes
id	SERIAL PK	
solution_id	INTEGER FK	→ solutions
user_id	INTEGER FK	→ users
value	SMALLINT	+1 or -1 only
created_at	TIMESTAMPTZ	
—	UNIQUE	(solution_id, user_id)

groups

Column	Type	Notes
id	SERIAL PK	
problem_id	INTEGER FK	→ problems
created_by	INTEGER FK	→ users
name	VARCHAR(150)	
description	TEXT	
created_at	TIMESTAMPTZ	

group_members

Column	Type	Notes
id	SERIAL PK	
group_id	INTEGER FK	→ groups
user_id	INTEGER FK	→ users
joined_at	TIMESTAMPTZ	

Column	Type	Notes
—	UNIQUE	(group_id, user_id)

notifications

Column	Type	Notes
id	SERIAL PK	
user_id	INTEGER FK	→ users
type	VARCHAR(50)	e.g. new_solution
reference_id	INTEGER	Points to relevant record
message	TEXT	
is_read	BOOLEAN	Default false
created_at	TIMESTAMPTZ	

API Endpoints

Auth — /api/auth

Method	Endpoint	Auth	Description
POST	/register	No	Create account (email, password, username, language)
POST	/login	No	Login, returns JWT
GET	/me	Yes	Get logged-in user's profile

Problems — /api/problems

Method	Endpoint	Auth	Description
GET	/	No	List all problems (filter by scope, location)
GET	/:id	No	Get one problem with solutions sorted by score

Method	Endpoint	Auth	Description
POST	/	Yes	Post a new problem
DELETE	/:id	Yes	Delete own problem

Solutions — /api/problems/:id/solutions

Method	Endpoint	Auth	Description
POST	/	Yes	Propose a solution to a problem
POST	/:solutionId/vote	Yes	Upvote (+1) or downvote (-1)
DELETE	/:solutionId	Yes	Delete own solution

Groups — /api/groups (phase 6)

Method	Endpoint	Auth	Description
POST	/	Yes	Create a group linked to a problem
GET	/:id	No	Get group details and members
POST	/:id/join	Yes	Join a group
POST	/:id/leave	Yes	Leave a group

Notifications — /api/notifications (phase 6)

Method	Endpoint	Auth	Description
GET	/	Yes	Get unread notifications for logged-in user
PUT	/:id/read	Yes	Mark a notification as read

Frontend Pages

Page	Route	Auth required
Home	/	No
Browse problems	/problems	No
Problem detail	/problems/:id	No
Post a problem	/problems/new	Yes
Propose a solution	/problems/:id/solutions/new	Yes
Register	/register	No
Login	/login	No
Groups (phase 6)	/groups/:id	No
Export report (phase 7)	/problems/:id/export	No

Frontend Components

Component	Location	Description
Navbar	components/Navbar.jsx	Sticky top nav, shows auth state
SolutionCard	components/SolutionCard.jsx	Solution with upvote/downvote buttons
AuthContext	context/AuthContext.jsx	Global login state and token

---

Build Phases

Phase 1 — Project setup ✅

- Node.js, React + Vite, Express, PostgreSQL installed and connected
- Frontend runs on localhost:5173
- Backend runs on localhost:3000

Phase 2 — Database design ✅

- All 7 tables created with foreign keys, indexes, and constraints
- ENUM type for scope (local/national)
- UNIQUE constraints on votes and group_members

Phase 3 — Auth ✅

- POST /api/auth/register — bcrypt password hashing, JWT returned
- POST /api/auth/login — credential check, JWT returned
- Auth middleware protects private routes
- React signup form with language selector
- AuthContext provides login state across the app

Phase 4 — Problems ✅

- POST, GET (all), GET (one), DELETE endpoints
- Scope and location filtering
- React browse page, detail page, and post form

Phase 5 — Solutions & voting ✅

- POST solution to a problem
- Upvote/downvote with toggle and vote-change logic
- Score stored on solutions table, updated on each vote
- Users cannot vote on their own solutions
- SolutionCard component with live score updates
- Navbar added across all pages

Phase 6 — Groups & notifications (in progress)

- Create a group linked to a problem
- Open groups — anyone can join
- Notify all group members when a new solution is posted
- Notification bell in navbar with unread count

Phase 7 — Export & share (planned)

- Export any problem page as a PDF report
- Report includes: problem, all solutions, vote scores
- Social share buttons: WhatsApp, Twitter/X, Facebook

Phase 8 — Languages & deploy (planned)

- i18next for UI translations: English, French, Spanish
- Language set automatically from user's preferred_language
- Frontend deployed to Vercel
- Backend + database deployed to Railway

Key Design Decisions

- **Score stored on solutions** — updated on each vote rather than calculated on the fly, keeps problem pages fast at scale
- **UNIQUE database constraints** — votes and group memberships are deduplicated at the database level, not just in app code
- **Generic login error messages** — login always returns "Invalid email or password" regardless of which field is wrong, preventing user enumeration
- **Public browsing** — anyone can read problems and solutions without signing up, lowering the barrier to discovery
- **Open groups** — any member can join any group, keeping the platform accessible and maximising participation
- **JWT stored in localStorage** — user stays logged in across page refreshes; token expires after 7 days

---

Folder Structure

```
my-platform/
├── backend/
|   ├── middleware/
|   |   └── auth.js
|   ├── routes/
```

```
│   │   ├── auth.js
│   │   ├── problems.js
│   │   └── solutions.js
│   ├── db.js
│   ├── index.js
│   └── .env
└── frontend/
    └── src/
        ├── components/
        │   ├── Navbar.jsx
        │   └── SolutionCard.jsx
        ├── context/
        │   └── AuthContext.jsx
        ├── pages/
        │   ├── HomePage.jsx
        │   ├── LoginPage.jsx
        │   ├── RegisterPage.jsx
        │   ├── ProblemsPage.jsx
        │   ├── ProblemDetailPage.jsx
        │   ├── PostProblemPage.jsx
        │   └── PostSolutionPage.jsx
        ├── App.jsx
        └── main.jsx
```